

Assignment No.

06:

Finite State

Machines (FSMs)

1. Design and simulate a sequence detector for detecting '1001'.

a) Draw the Mealy FSM state diagram.

b) Implement the Mealy FSM in Verilog HDL.

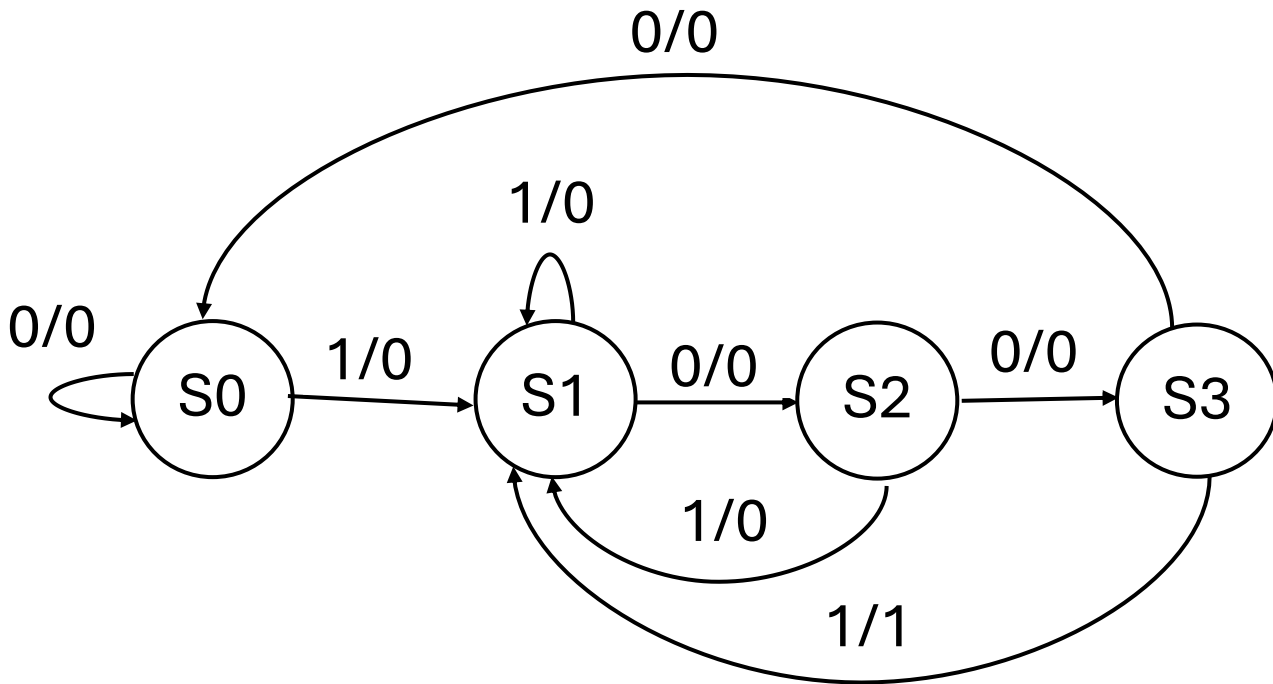
c) Draw the Moore FSM state diagram

d) Implement the Moore FSM in Verilog HDL.

e) Compare Mealy and Moore FSM implementations.

a)

Mealy FSM state diagram



b)

```
module mealy_1001_detector (  
    input clk,  
    input rst_n,  
    input x,  
    output reg z);  
  
    typedef enum reg [1:0] {  
        S0 = 2'b00,  
        S1 = 2'b01,  
        S2 = 2'b10,  
        S3 = 2'b11
```

```

} state_t;

state_t current_state, next_state;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n)
        current_state <= S0;
    else
        current_state <= next_state;
end

always @(*) begin

    next_state = current_state;
    z = 1'b0;

    case (current_state)
        S0: begin
            if (x) next_state = S1;
            else   next_state = S0;
        end

        S1: begin
            if (x) next_state = S1;
            else   next_state = S2;
        end

        S2: begin
            if (x) next_state = S1;
            else   next_state = S3;
        end

        S3: begin
            if (x) begin
                next_state = S1;
                z = 1'b1;
            end else begin
                next_state = S0;
            end
        end

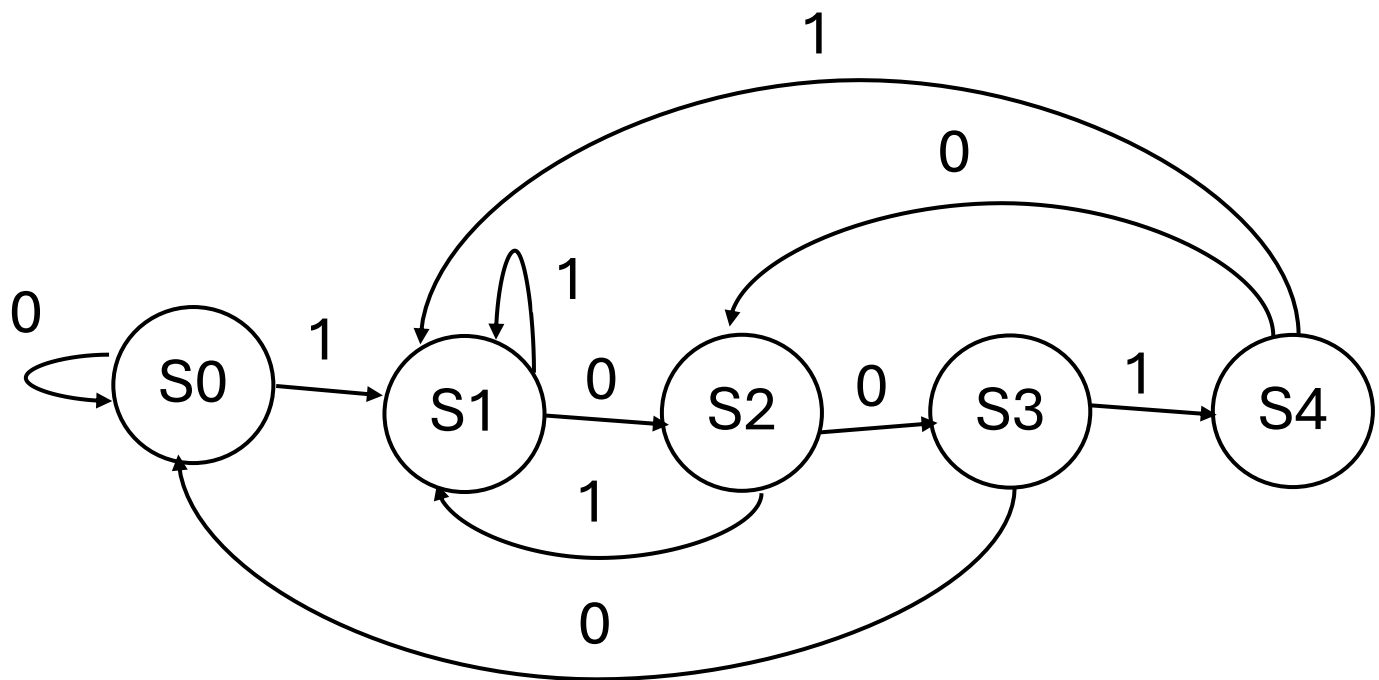
        default: next_state = S0;
    endcase
end

endmodule

```

c)

Moore FSM state diagram



```
d) module moore_1001_detector (  
    input clk,  
    input rst_n,  
    input x,  
    output reg z);  
  
    typedef enum reg [2:0] {  
        S0 = 3'b000,  
        S1 = 3'b001,  
        S2 = 3'b010,  
        S3 = 3'b011,  
        S4 = 3'b100  
    } state_t;  
  
    state_t current_state, next_state;  
  
    always @(posedge clk or negedge rst_n) begin  
        if (!rst_n)  
            current_state <= S0;  
        else  
            current_state <= next_state;  
    end  
  
    always @(*) begin
```

```
next_state = current_state;
```

```
case (current_state)
```

```
  S0: begin
```

```
    if (x) next_state = S1;
```

```
    else  next_state = S0;
```

```
  end
```

```
  S1: begin
```

```
    if (x) next_state = S1;
```

```
    else  next_state = S2;
```

```
  end
```

```
  S2: begin
```

```
    if (x) next_state = S1;
```

```
    else  next_state = S3;
```

```
  end
```

```
  S3: begin
```

```
    if (x) next_state = S4;
```

```
    else  next_state = S0;
```

```
  end
```

```
  S4: begin
```

```
    if (x) next_state = S1;
```

```
    else  next_state = S2; // Overlapping behavior: '10' matched
```

```
  end
```

```
  default: next_state = S0;
```

```
endcase
```

```
end
```

```
always @(*) begin
```

```
  z = (current_state == S4) ? 1'b1 : 1'b0;
```

```
end
```

```
endmodule
```

e) Comparing Mealy and Moore FSM Implementations

Feature	Mealy FSM	Moore FSM
State Count	Typically fewer states (4 states for '1001').	Typically more states (5 states for '1001').
Output Dependency	Depends on both the current state and the input .	Depends only on the current state .
Output Timing	Responds immediately to input changes within the clock cycle (asynchronous output spikes possible).	Output is synchronized with the clock edge; changes occur when transitioning into a state.
Glitch Susceptibility	High. Input noise or transients can cause glitches on the output.	Low. Outputs are buffered and clean because they do not change with input changes alone.
Design Complexity	Slightly more complex output logic.	Easier to write, debug, and trace state transitions.